

Infrastructure as Code (IaC)

Streamlining Infrastructure Management

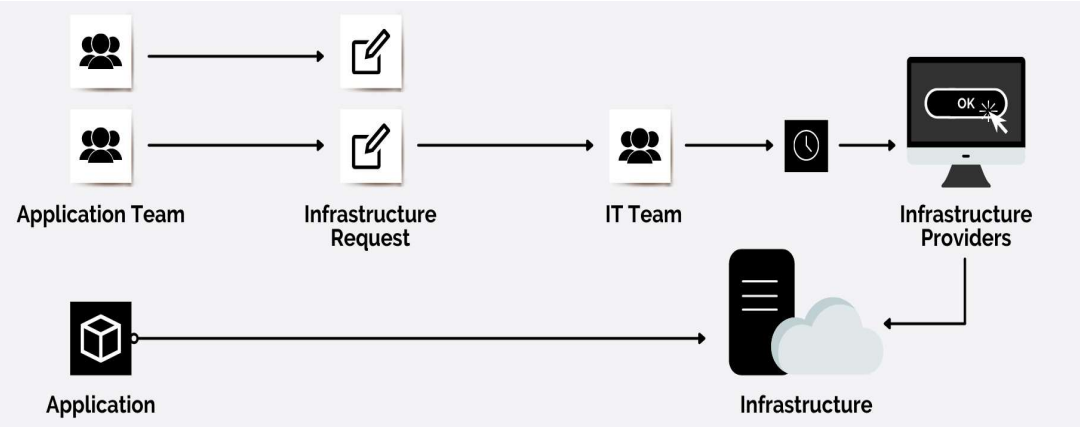
Agenda

- *Why IaC?*
- *What is IaC?*
- *Brief History*
- *How does IaC works?*
- *IaC Tools*
- *Key Concepts of IaC*
- *Benefits*
- *IaC with Terraform*
- *Terraform features*
- *Terraform workflow*
- *How does terraform works?*
- *Different Types of Blocks in Terraform*
- *Hands-on Lab*



Why IaC?

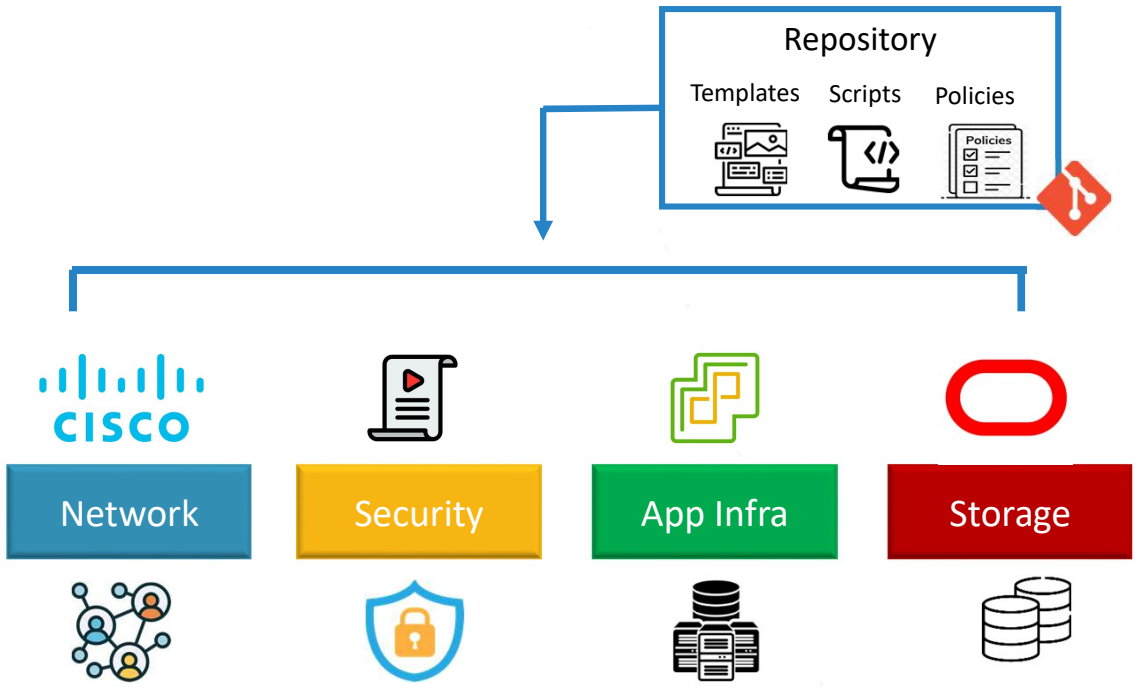
Traditional infrastructure deployment



- Manual and time-consuming process
- Error-prone
- Inconsistency
- Configuration drift
- Difficulty to keep multiple environments in lockstep
- Scalability
- Difficult to document

What is IaC?

Infrastructure as code is a modern approach to managing and provisioning infrastructure resources using code and automation. Instead of configuring and managing infrastructure manually, IaC allows you to define your infrastructure, including servers, networks, databases, and other resources, through code.



History

Early 2000s: Emergence of Automation Tools

- The need for efficient and scalable infrastructure management led to the emergence of automation tools in the early 2000s.
- Tools like Puppet and Chef gained popularity for automating configuration management tasks.

Mid-2000s: Rise of Virtualization

- The mid-2000s saw a significant shift towards virtualization technologies.
- Virtualization allowed for the abstraction of physical hardware, enabling more flexible and scalable infrastructure.

Late 2000s: Configuration Management Tools

- Configuration management tools like Puppet and Chef continued to evolve, providing a way to define and manage infrastructure as code.

2010: DevOps Movement

- The DevOps movement gained momentum, emphasizing collaboration between development and operations teams.

2014: Launch of Terraform

- HashiCorp launched Terraform in 2014, introducing a declarative approach to infrastructure provisioning and management.
- Terraform allowed users to define infrastructure as code using HashiCorp Configuration Language (HCL).

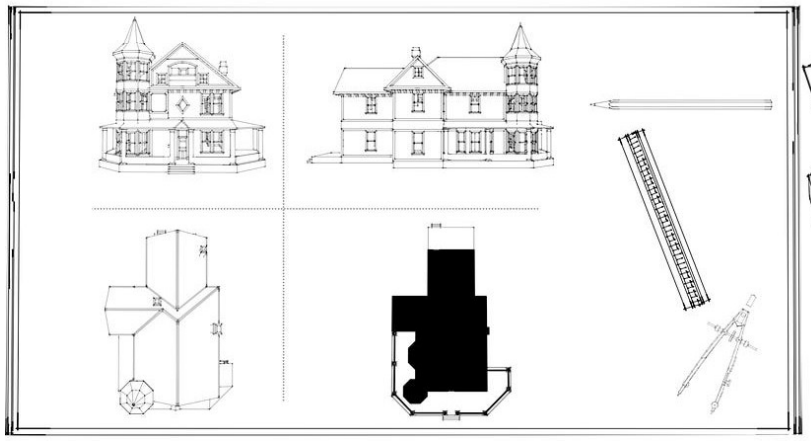
2017: Cloud-Native Development

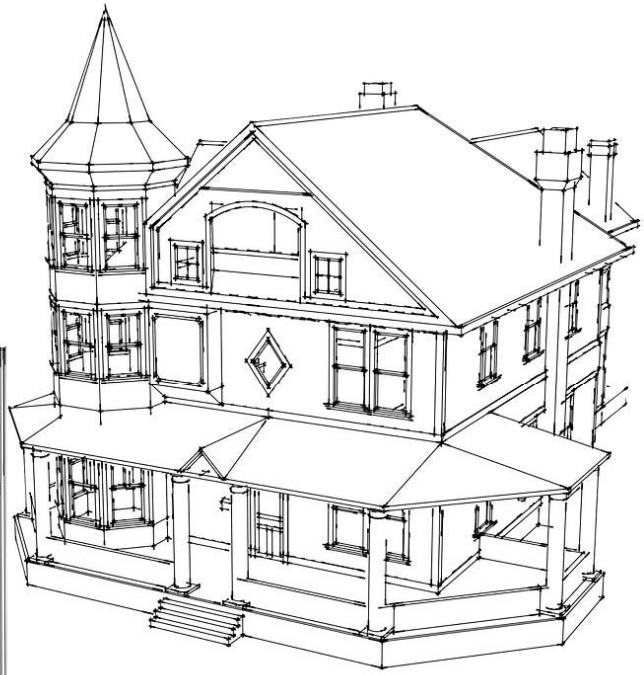
- With the rise of cloud-native development, IaC became even more crucial for managing dynamic and scalable cloud infrastructure.
- IaC tools like Terraform, Ansible, and others became essential components of cloud orchestration and deployment.

How does IaC Works?

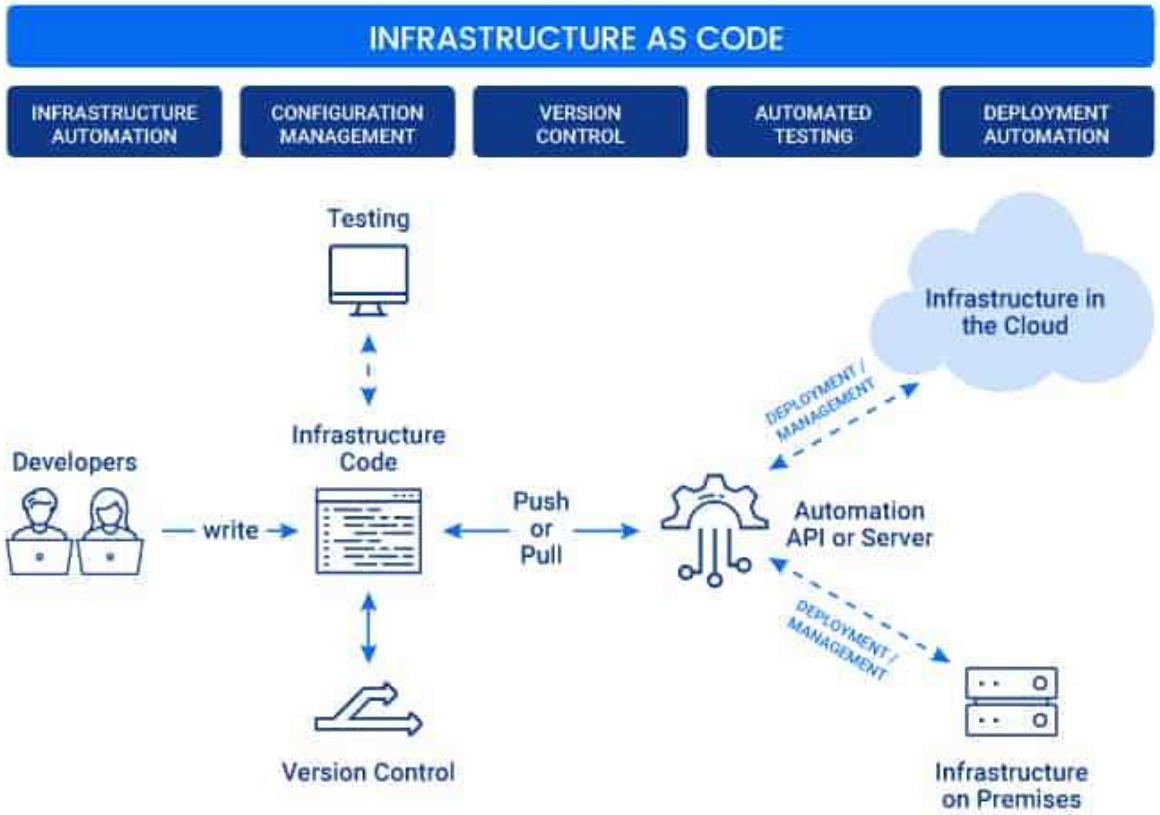
It outlines the layout, dimensions, materials, and various components of the structure

Architecture Blueprint





Constructed House


INFRASTRUCTURE AS CODE
INFRASTRUCTURE AUTOMATION
CONFIGURATION MANAGEMENT
VERSION CONTROL
AUTOMATED TESTING
DEPLOYMENT AUTOMATION
Testing
Infrastructure in the Cloud
DEPLOYMENT / MANAGEMENT
Infrastructure Code
Developers
write
Push or Pull
Automation API or Server
DEPLOYMENT / MANAGEMENT
Version Control
Infrastructure on Premises

IaC Tools

- Azure Resource Manager (Azure)
- CloudFormation (AWS)
- Deployment manager (GCP)
- Terraform



Key Concept

Imperative

what to do

- Forward 1 mile
- Turn right
- Forward 2 miles
- Turn left
- Forward 3 miles
- Arrive at pizza restaurant



Declarative

what is desired

- Go to the pizza restaurant





Imperative

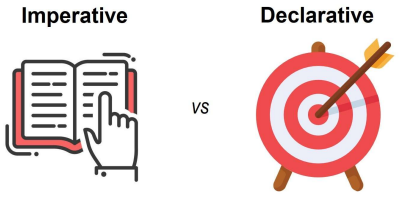
what to do

- Starting point is crucial
- Audit Complexity
- Drift Detection Challenges
- Lack of version control hinders the ability to track changes and revert to previous states
- Replicating processes becomes problematic, leading to non-repeatable outcomes
- Replicating processes becomes problematic, leading to non-repeatable outcomes
- Changes to destination requires significant modifications

Declarative

what is desired

- Starting point does not matter
- Engine determines how to get to destination
- Self-consistent
- Repeatable in a pipeline
- Easy to validate and detect drift
- Can be version controlled
- Changes to destination automatically handled



Benefits

Version Control & Collaboration



IaC configurations are treated as code and can be stored in version control systems (e.g., Git). This enables collaboration among teams and tracks changes over time.

Speed & Efficiency



Provisioning resources manually can take hours or days. IaC drastically reduces this time to minutes, accelerating the deployment of applications and services

Cost Savings



IaC enables efficient resource allocation, ensuring that resources are provisioned only as needed. This leads to reduced over-provisioning and lower cloud infrastructure costs

Consistency and Reliability



IaC enforces consistent infrastructure configurations across different environments, reducing discrepancies between different environments

Security and Compliance



IaC allows organizations to define security settings and compliance policies as code. This ensures that all resources are provisioned with the necessary security measures.

IaC with Terraform

Terraform is one of the most popular **Infrastructure-as-code (IaC) tool**, used by DevOps teams to automate infrastructure tasks. It is used to automate the provisioning of your cloud resources. Terraform is an open-source, cloud-agnostic provisioning tool developed by HashiCorp and written in GO language.

Terraform Features

1

Infrastructure as Code (IaC)
Terraform embraces the IaC methodology, allowing you to define infrastructure configurations using code.

2

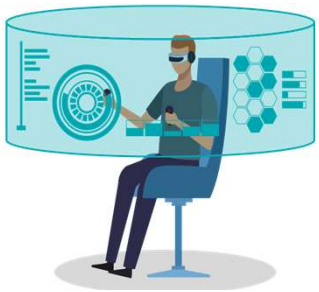
Declarative Syntax
Terraform uses a declarative language to define the desired state of your infrastructure.

3

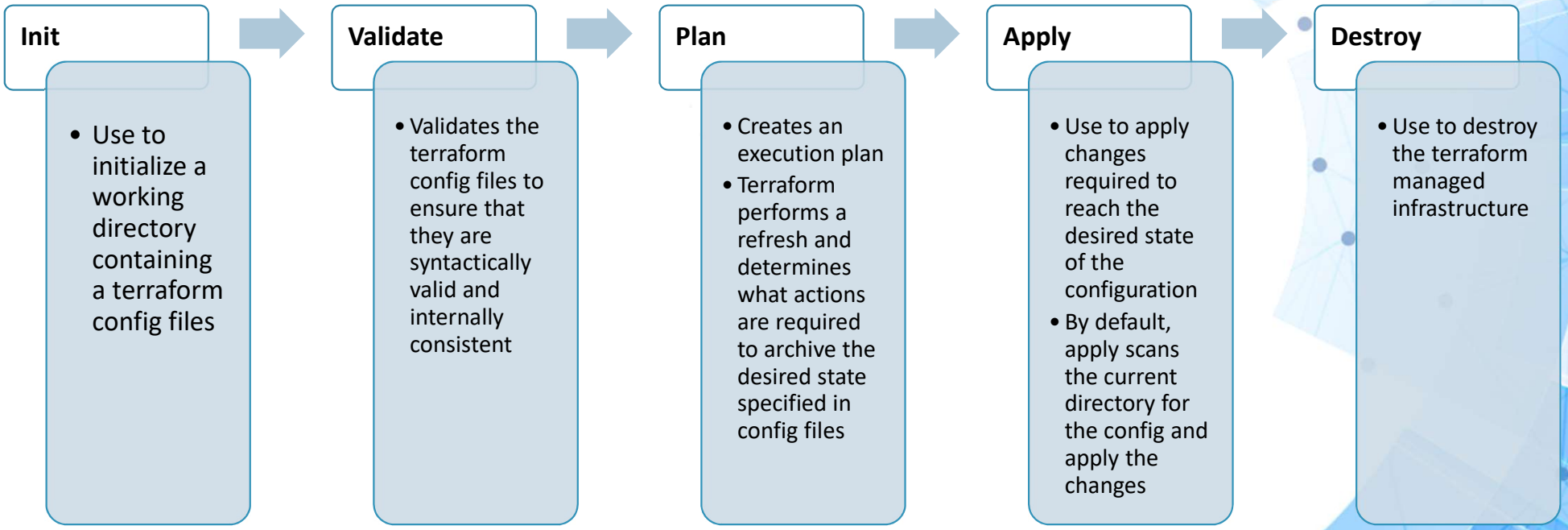
State Management
Terraform maintains a state file that records the current state of your infrastructure.

4

Parallelism and Efficiency
Terraform can provision and update resources in parallel, improving performance and reducing the time needed for large-scale deployments

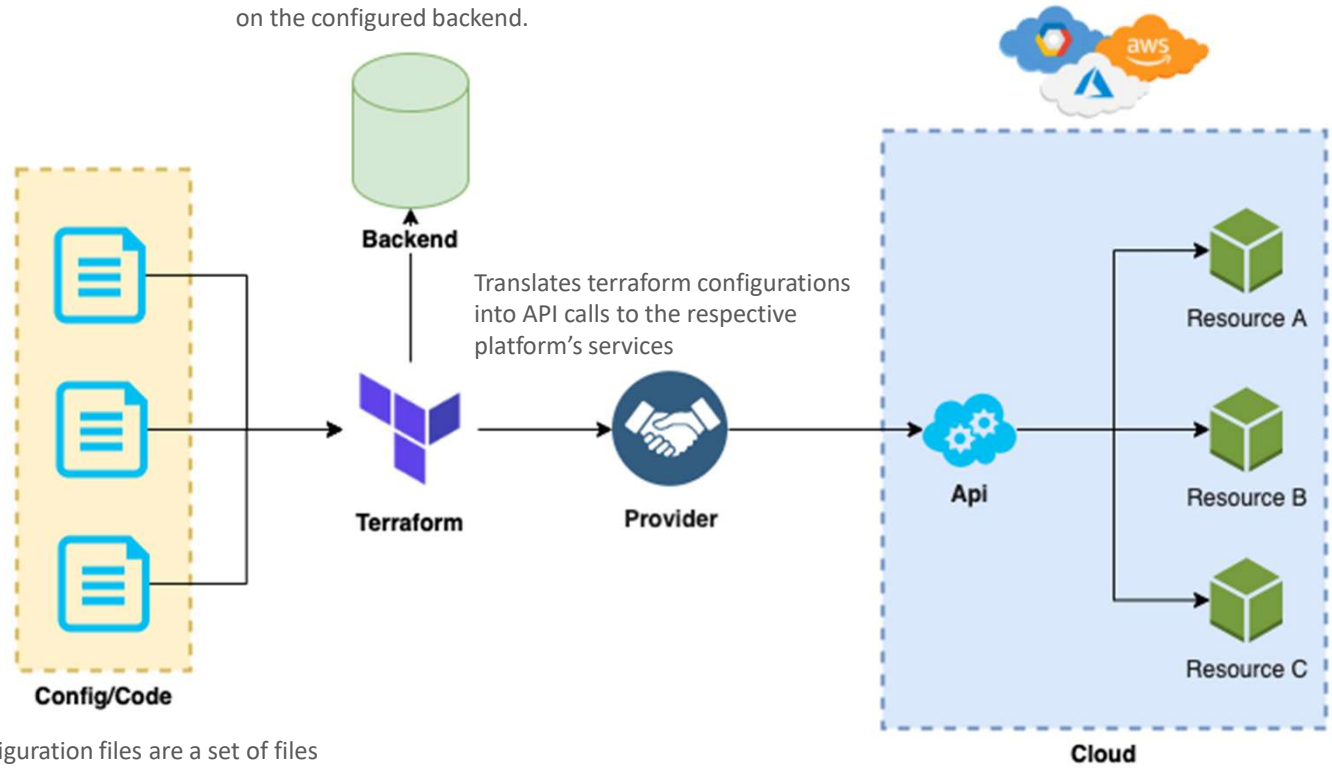


Terraform Workflow



How terraform works?

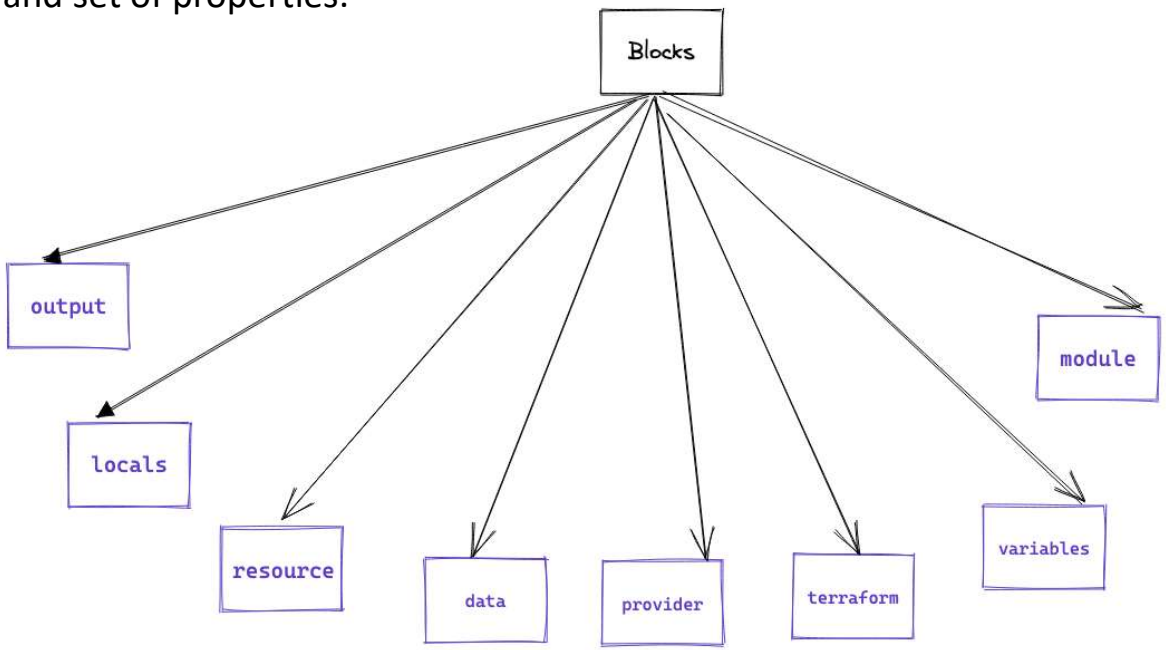
Holds information about the current infrastructure in a state file on the configured backend.



Configuration files are a set of files used to describe infrastructure in Terraform and have the file extensions .tf and .tf.json.

Different Types of Blocks in Terraform

In Terraform, a block is a fundamental unit used to define and configure different aspects of your infrastructure. Blocks are written in HashiCorp Configuration Language (HCL) and allow you to declare resources, providers, variables, outputs, and other configuration elements within your Terraform code. Each block serves a specific purpose and has its own syntax and set of properties.



Terraform

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }}}
provider "aws" {
  region = "ap-south-1" # Choose a nearby AWS region
}
locals {
  instance_name = "my-simple-server"
  instance_type = "t2.micro"
}
resource "aws_instance" "server" {
  ami           = "ami-xxxxxxxxxxxxxxxxx" # Replace with a valid AMI ID for your region
  instance_type = local.instance_type
  tags = {
    Name = local.instance_name
  }}
output "public_ip" {
  value       = aws_instance.server.public_ip
  description = "Public IP of the EC2 instance"
}
```